

Veritas

AI-Powered Misinformation Detection System

BSc Data Science - Third Year Project

Source Code Documentation

August 2026

Contents

01. `src/convex/analyzeNews.ts`

NLP-based fake news detection engine

02. `src/convex/schema.ts`

Database schema for analysis storage

03. `src/convex/analyses.ts`

Backend queries and mutations

04. `src/components/CredibilityGauge.tsx`

Animated SVG confidence gauge

05. `src/components/StatsView.tsx`

Statistics dashboard with charts

06. `src/components/MethodologyView.tsx`

Academic methodology documentation

```

1  "use node";
2
3  import { action } from "../generated/server";
4  import { v } from "convex/values";
5
6  // --- Enhanced NLP-Based Fake News Detection Engine v2 -----
7  // Multi-layered linguistic analysis with severity-weighted scoring,
8  // domain-specific heuristics, and structural pattern detection.
9
10 // --- WORD LISTS -----
11
12 const SENSATIONALIST: Array<RegExp, number> = [
13   [/[A-Z]{3,}!{2,}/g, 8], // EXPOSED!!!
14   [/shocking|unbelievable|mind[- ]?blowing/gi, 6],
15   [/miracle|wonder|amazing|incredible/gi, 5],
16   [/urgent|breaking|just in/gi, 4],
17   [/exposed|revealed|exposed!!!/gi, 6],
18   [/secret|hidden|suppressed|banned|censored/gi, 6],
19   [/cover[- ]?up|conspiracy|conspiracy!!!/gi, 7],
20   [/they don't want you to know/gi, 8],
21   [/share (this|before|now)|before they delete/gi, 7],
22   [/wake up|open your eyes|do your research/gi, 6],
23   [/the truth (about|they|is)|what they're hiding/gi, 7],
24   [/mainstream media (won't|doesn't|is paid)/gi, 8],
25   [/explosive|bombshell|devastating truth/gi, 5],
26   [/paradigm shift|game[- ]?changer/gi, 4],
27   [/you won't (believe|want to miss)/gi, 7],
28 ];
29
30 const CLICKBAIT: Array<RegExp, number> = [
31   [/you won't believe/i, 6],
32   [/doctors? (don't|hate|are shocked)/i, 7],
33   [/one|a) (trick|simple|weird|secret)/i, 5],
34   [/the|this) one (trick|simple|weird)/i, 6],
35   [/number \d+ will (shock|amaze|surprise)/i, 7],
36   [/what happens (next|when)/i, 5],
37   [/gone wrong/i, 4],
38   [/^d+%s*of\s*(people|doctors|scientists)\s*(don't|won't)/i, 7],
39   [/before it's|while you still can)/i, 5],
40   [/click here|act now|limited time)/i, 6],
41   [/this simple|one weird) (fix|solution|trick)/i, 6],
42   [/celebrities|hate|love) this (trick|secret|hack)/i, 7],
43 ];
44
45 const ANONYMOUS_SOURCING: Array<RegExp, number> = [
46   [/experts? (say|claim|believe|warn|say)/gi, 4],
47   [/studies (show|reveal|suggest|prove)/gi, 3],
48   [/scientists? (say|warn|discover|reveal)/gi, 3],
49   [/insiders? (reveal|say|claim)/gi, 5],
50   [/sources? (say|claim|reveal)/gi, 4],
51   [/researchers? (say|warn|find)/gi, 3],
52   [/a|the) (senior|top|high[- ]?ranking|unnamed)/gi, 4],
53   [/anonymous (source|insider|official)/gi, 7],
54   [/people|everyone) (are saying|say|claim)/gi, 5],
55   [/they|people) are saying/gi, 4],
56   [/a (certain|well[- ]?known|highly placed)/gi, 5],
57   [/our sources|inside sources|multiple sources)/gi, 5],
58 ];
59
60 const FEAR_MONGERING: Array<RegExp, number> = [
61   [/the elite|the powerful|the rich) (are|want|plan|will) to/gi, 5],
62   [/big (pharma|tech|media|tobacco|oil)/gi, 6],
63   [/deep state|new world order|illuminati/gi, 7],
64   [/mind control|brainwash|programming/gi, 6],
65   [/poison(ing|ed|s)?|toxic|chemical(ly)?/gi, 5],

```

```

66     [/depopulation|eugenics|population control/gi, 7],
67     [/end times|apocalypse|armageddon/gi, 5],
68     [/doom(ed)?|catastroph(e|ic)/gi, 4],
69     [/((prepare|get ready) for (the worst|war|collapse)/gi, 5],
70     [/((global|worldwide) (collapse|crisis|emergency)/gi, 4],
71     [/((your children|future generations|we all)/gi, 3],
72 ];
73
74 const CONSPIRACY: Array<[RegExp, number]> = [
75     [/conspiracy|conspir(acy|ies|ing|atorial)/gi, 6],
76     [/cover[- ]?up|covering up/gi, 6],
77     [/wake up|open your eyes/gi, 6],
78     [/do your (own )?research/gi, 5],
79     [/sheeple|wake[- ]?up sheeple/gi, 7],
80     [/fake news|lamestream|lame[- ]?stream media/gi, 5],
81     [/deep state|shadow government/gi, 7],
82     [/globalist|global elites|cabal/gi, 6],
83     [/((the truth|what they|the real)/gi, 4],
84     [/mainstream media (is|are|won't)/gi, 5],
85     [/((google|youtube|facebook|twitter) (is|are) censoring/gi, 6],
86     [/censored|silenced|suppressed(?!.*science)/gi, 5],
87 ];
88
89 const CREDIBLE_INDICATORS: Array<[RegExp, number]> = [
90     [/according to (the |a |research |official )/gi, 5],
91     [/published (in|on|by) (the |a )?/gi, 6],
92     [/((study|research|report|analysis) (published|found|conducted|released)/gi, 6],
93     [/((data|statistics|findings) (from|show|indicate|suggest)/gi, 5],
94     [/((officials?|spokesperson|representative) (said|stated|confirmed|announced)/gi, 5],
95     [/in a (press|public|official) statement/gi, 6],
96     [/((CEO|CTO|CFO|president|director|minister|secretary|chairman|dean)/gi, 4],
97     [/((on|during|at) (monday|tuesday|wednesday|thursday|friday|saturday|sunday)/gi, 4],
98     [/((january|february|march|april|may|june|july|august|september|october|november|december)\s+\\.\\.\\.
99     [/\d{1,2}\.\/\d{1,2}\.\/\d{2,4}/g, 4],
100    [/((peer[- ]?reviewed|peer reviewed)/gi, 7],
101    [/((according to|as reported by|citing|cited)/gi, 4],
102    [/((found that|showed that|revealed that|confirmed that)/gi, 4],
103    [/((significant|statistically significant|notable)/gi, 3],
104 ];
105
106 const CREDIBLE_SOURCES = [
107     /\breuters\b/i, /\bap news\b/i, /\bassociated press\b/i,
108     /\bbbc\b/i, /\bbc news\b/i, /\bnew york times\b/i,
109     /\bwashington post\b/i, /\bthe guardian\b/i, /\bcnn\b/i,
110     /\bnbc\b/i, /\bcbs\b/i, /\babc news\b/i, /\bnpr\b/i,
111     /\bnature\b/i, /\bscience\b/i, /\bthe lancet\b/i,
112     /\bjournal of\b/i, /\buniversity of\b/i, /\binstitute\b/i,
113     /\bfederal reserve\b/i, /\bworld health organization\b/i,
114     /\bnasa\b/i, /\bnoaa\b/i, /\bcdcb\b/i, /\bfda\b/i,
115     /\breuters institute\b/i, /\broyal society\b/i,
116     /\bamerican medical association\b/i, /\bhharvard\b/i,
117     /\bmmt\b/i, /\bstanford\b/i, /\boxford\b/i, /\bcambridge\b/i,
118 ];
119
120 // --- ANALYSIS FUNCTIONS -----
121
122 function weightedMatches(text: string, patterns: Array<[RegExp, number]>): { total: number; sco...
123     let total = 0;
124     let score = 0;
125     const matches: string[] = [];
126
127     for (const [pattern, weight] of patterns) {
128         const found = text.match(pattern);
129         if (found) {
130             total += found.length;
131             score += found.length * weight;
132             matches.push(...found.slice(0, 2).map((m) => m.trim()));
133         }
134     }
135     return { total, score, matches: [...new Set(matches)].slice(0, 3) };

```

```
136 }
137
138 function countMatches(text: string, patterns: RegExp[]): number {
139     return patterns.reduce((c, p) => c + (text.match(p)?.length ?? 0), 0);
140 }
141
142 interface AnalysisResult {
143     verdict: "likely_real" | "likely_fake" | "uncertain";
144     confidence: number;
145     summary: string;
146     redFlags: string[];
147     greenFlags: string[];
148     reasoning: string;
149 }
150
151 function analyzeText(text: string): AnalysisResult {
152     const words = text.split(/\s+/);
153     const wordCount = words.length;
154     const sentences = text.split(/[.!?]+/).filter((s) => s.trim().length > 0);
155     const sentenceCount = sentences.length;
156
157     let redFlagScore = 0;
158     let greenFlagScore = 0;
159     const redFlags: string[] = [];
160     const greenFlags: string[] = [];
161
162     // --- RED FLAG ANALYSIS (weighted scoring) -----
163
164     // 1. Sensationalist language
165     const sens = weightedMatches(text, SENSATIONALIST);
166     if (sens.score >= 30) {
167         redFlagScore += 30;
168         redFlags.push(`Highly sensationalist language (${sens.total} instances: "${sens.matches.sli...`);
169     } else if (sens.score >= 15) {
170         redFlagScore += 18;
171         redFlags.push(`Sensationalist language detected (${sens.total} instances)`);
172     } else if (sens.score >= 5) {
173         redFlagScore += 8;
174         redFlags.push("Mild sensationalist tone detected");
175     }
176
177     // 2. Clickbait patterns
178     const cb = weightedMatches(text, CLICKBAIT);
179     if (cb.score >= 15) {
180         redFlagScore += 25;
181         redFlags.push(`Multiple clickbait patterns detected (${cb.total} matches)`);
182     } else if (cb.score >= 8) {
183         redFlagScore += 14;
184         redFlags.push(`Clickbait pattern detected: "${cb.matches[0]}"`);
185     } else if (cb.score >= 3) {
186         redFlagScore += 6;
187         redFlags.push("Possible clickbait language");
188     }
189
190     // 3. Anonymous / vague sourcing
191     const anon = weightedMatches(text, ANONYMOUS_SOURCING);
192     if (anon.score >= 20) {
193         redFlagScore += 22;
194         redFlags.push("Heavy reliance on anonymous or vague sourcing");
195     } else if (anon.score >= 10) {
196         redFlagScore += 12;
197         redFlags.push("Some anonymous or vague sourcing detected");
198     } else if (anon.score >= 4) {
199         redFlagScore += 5;
200         redFlags.push("Minor vague sourcing detected");
201     }
202
203     // 4. Fear-mongering
204     const fear = weightedMatches(text, FEAR_MONGERING);
205     if (fear.score >= 20) {
```

```
206     redFlagScore += 22;
207     redFlags.push("Fear-mongering and alarmist language detected");
208   } else if (fear.score >= 10) {
209     redFlagScore += 12;
210     redFlags.push("Some alarmist language detected");
211   } else if (fear.score >= 4) {
212     redFlagScore += 5;
213     redFlags.push("Mild alarmist undertones");
214   }
215
216   // 5. Conspiracy language
217   const consp = weightedMatches(text, CONSPIRACY);
218   if (consp.score >= 18) {
219     redFlagScore += 28;
220     redFlags.push(`Conspiracy theory language detected (${consp.total} markers)`);
221   } else if (consp.score >= 9) {
222     redFlagScore += 16;
223     redFlags.push("Conspiracy-themed language detected");
224   } else if (consp.score >= 4) {
225     redFlagScore += 7;
226     redFlags.push("Some conspiratorial framing");
227   }
228
229   // 6. Excessive caps / exclamation marks
230   const capsMatches = text.match(/[A-Z]{3,}!{1,}/g);
231   const capsCount = capsMatches?.length ?? 0;
232   if (capsCount >= 5) {
233     redFlagScore += 18;
234     redFlags.push(`Excessive capitalization and exclamation marks (${capsCount} instances)`);
235   } else if (capsCount >= 2) {
236     redFlagScore += 8;
237     redFlags.push("Some excessive capitalization or exclamation marks");
238   }
239
240   // 7. Emoji usage (unprofessional)
241   const emojiCount = (text.match(/[\u{1F600}-\u{1F64F}|\u{1F300}-\u{1F5FF}|\u{1F680}-\u{1F6FF}|\u{...
242   if (emojiCount >= 3) {
243     redFlagScore += 12;
244     redFlags.push(`Excessive emoji usage (${emojiCount} emojis - uncommon in journalism)`);
245   } else if (emojiCount >= 1) {
246     redFlagScore += 4;
247     redFlags.push("Emoji usage detected (uncommon in credible journalism)");
248   }
249
250   // 8. ALL CAPS words
251   const allCapsWords = text.match(/\b[A-Z]{3,}\b/g);
252   const allCapsCount = allCapsWords?.filter(w => ![ "CEO", "CTO", "NASA", "FDA", "CDC", "WHO", ...
253   if (allCapsCount >= 6) {
254     redFlagScore += 14;
255     redFlags.push(`Excessive use of ALL CAPS words (${allCapsCount} instances)`);
256   } else if (allCapsCount >= 3) {
257     redFlagScore += 7;
258     redFlags.push("Some ALL CAPS emphasis detected");
259   }
260
261   // 9. Multiple consecutive exclamation marks
262   const multiExcl = text.match(/!{3,}/g);
263   if (multiExcl && multiExcl.length >= 2) {
264     redFlagScore += 10;
265     redFlags.push("Multiple sets of consecutive exclamation marks");
266   }
267
268   // 10. "Share before deleted" urgency
269   const urgency = text.match(/share.*(before|now|this)|before.*(delete|remove|censor)|while you...
270   if (urgency) {
271     redFlagScore += 12;
272     redFlags.push("Urgency language designed to pressure sharing");
273   }
274
275   // 11. Lack of any URL or source citation
```

```

276     const hasUrl = /https?:\/\/|www\.|doi\.org|arxiv\.org/i.test(text);
277     const hasSourceCitation = countMatches(text, [
278         /according to/i, /published in/i, /cited by/i, /source:/i,
279         /the (report|study|paper) (from|by|at)/i,
280     ]);
281     if (!hasUrl && hasSourceCitation === 0 && wordCount > 40) {
282         redFlagScore += 6;
283     }
284
285     // 12. Short article length
286     if (wordCount < 25 && wordCount > 5) {
287         redFlagScore += 4;
288     }
289
290     // --- GREEN FLAG ANALYSIS -----
291
292     // 1. Named sources with attributions
293     const namedSources = weightedMatches(text, CREDIBLE_INDICATORS);
294     if (namedSources.score >= 25) {
295         greenFlagScore += 25;
296         greenFlags.push(`Strong sourcing and attribution (${namedSources.total} indicators)`);
297     } else if (namedSources.score >= 12) {
298         greenFlagScore += 16;
299         greenFlags.push(`Good sourcing and attribution (${namedSources.total} indicators)`);
300     } else if (namedSources.score >= 5) {
301         greenFlagScore += 8;
302         greenFlags.push("Some sourcing and attribution present");
303     }
304
305     // 2. Specific statistics / data
306     const hasStats = /\d+%\d+ percent|\d+\.\d+|[?[\d,]+\|\$[\d,]+\|?[?[\d,]+\d+ million|\d+ billion|...
307     if (hasStats) {
308         greenFlagScore += 16;
309         greenFlags.push("Specific statistics and quantitative data cited");
310     }
311
312     // 3. Credible sources mentioned
313     const credibleCount = countMatches(text, CREDIBLE_SOURCES);
314     if (credibleCount >= 3) {
315         greenFlagScore += 22;
316         greenFlags.push(`Multiple references to credible institutions (${credibleCount} sources)`);
317     } else if (credibleCount >= 1) {
318         greenFlagScore += 12;
319         greenFlags.push("Reference to a credible institution or publication");
320     }
321
322     // 4. Dates and temporal specificity
323     const hasDate = /(?:january|february|march|april|may|june|july|august|september|october|novem...
324     if (hasDate) {
325         greenFlagScore += 14;
326         greenFlags.push("Specific dates and temporal references provided");
327     }
328
329     // 5. Balanced / nuanced reporting
330     const balanced = countMatches(text, [
331         /on the other hand/i, /however/i, /conversely/i, /in contrast/i,
332         /critics (say|argue|claim|point out)/i, /opponents/i, /supporters/i,
333         /while .* (argue|claim|believe)/i, /both (sides|perspectives)/i,
334         /some (experts|analysts|officials)/i, /others (believe|argue|say)/i,
335     ]);
336     if (balanced >= 3) {
337         greenFlagScore += 18;
338         greenFlags.push("Balanced reporting with multiple perspectives");
339     } else if (balanced >= 1) {
340         greenFlagScore += 10;
341         greenFlags.push("Some balanced perspectives present");
342     }
343
344     // 6. Journalistic structure
345     const hasWho = /\b(?:who|whom|whose)\b/i.test(text);

```

```

346 const hasWhat = /\b(?:what|which)\b/i.test(text);
347 const hasWhere = /\b(?:where)\b/i.test(text);
348 const hasWhen = /\b(?:when|while|during|after|before)\b/i.test(text);
349 const hasWhy = /\b(?:why|because|since|due to|as a result|accordingly)\b/i.test(text);
350 const structureCount = [hasWho, hasWhat, hasWhere, hasWhen, hasWhy].filter(Boolean).length;
351 if (structureCount >= 4) {
352   greenFlagScore += 16;
353   greenFlags.push("Follows journalistic structure (who, what, where, when, why)");
354 } else if (structureCount >= 3) {
355   greenFlagScore += 10;
356   greenFlags.push("Partial journalistic structure present");
357 } else if (structureCount >= 2) {
358   greenFlagScore += 5;
359   greenFlags.push("Basic informational structure detected");
360 }
361
362 // 7. Neutral / measured tone
363 const emotionalWords = countMatches(text, [
364   /amazing|terrible|horrible|wonderful|awful|disgusting|incredible|unbelievable|outrage|furio...
365 ]);
366 if (emotionalWords === 0 && wordCount > 40) {
367   greenFlagScore += 12;
368   greenFlags.push("Neutral and measured tone throughout");
369 } else if (emotionalWords <= 1 && wordCount > 40) {
370   greenFlagScore += 6;
371   greenFlags.push("Mostly neutral tone");
372 }
373
374 // 8. Proper article length
375 if (wordCount >= 80 && wordCount <= 800) {
376   greenFlagScore += 6;
377   greenFlags.push("Appropriate article length for news reporting");
378 } else if (wordCount >= 50) {
379   greenFlagScore += 3;
380 }
381
382 // 9. Quotes from named individuals
383 const namedQuotes = countMatches(text, [
384   /[""].*?[""] (?:said|stated|noted|explained|added|continued|wrote)/gi,
385   /(?:said|stated|noted|explained|added) [" "].*?[""]/gi,
386   /[A-Z][a-z]+ (?:said|stated|noted|explained|wrote) [":]/gi,
387 ]);
388 if (namedQuotes >= 2) {
389   greenFlagScore += 14;
390   greenFlags.push("Multiple quotes from named individuals");
391 } else if (namedQuotes >= 1) {
392   greenFlagScore += 8;
393   greenFlags.push("Quote from a named individual");
394 }
395
396 // --- VERDICT CALCULATION -----
397
398 const totalScore = Math.max(redFlagScore + greenFlagScore, 1);
399 const redRatio = redFlagScore / totalScore;
400 const greenRatio = greenFlagScore / totalScore;
401
402 let verdict: "likely_real" | "likely_fake" | "uncertain";
403 let confidence: number;
404
405 if (redRatio >= 0.65) {
406   verdict = "likely_fake";
407   confidence = Math.min(95, Math.round(55 + redRatio * 40));
408 } else if (greenRatio >= 0.65) {
409   verdict = "likely_real";
410   confidence = Math.min(95, Math.round(55 + greenRatio * 40));
411 } else if (redRatio > greenRatio + 0.1) {
412   verdict = "likely_fake";
413   confidence = Math.min(78, Math.round(42 + (redRatio - greenRatio) * 35));
414 } else if (greenRatio > redRatio + 0.1) {
415   verdict = "likely_real";

```

```

416     confidence = Math.min(78, Math.round(42 + (greenRatio - redRatio) * 35));
417   } else {
418     verdict = "uncertain";
419     confidence = Math.round(35 + Math.abs(redRatio - greenRatio) * 15);
420   }
421
422   confidence = Math.max(35, Math.min(95, confidence));
423
424   // --- SUMMARY GENERATION -----
425
426   const topRed = redFlags[0] ?? "language patterns raise concerns";
427   const topGreen = greenFlags[0] ?? "structural elements are present";
428   const redCount = redFlags.length;
429   const greenCount = greenFlags.length;
430
431   let summary: string;
432   if (verdict === "likely_fake") {
433     summary = `This content shows ${redCount} red flag${redCount !== 1 ? "s" : ""} suggesting i...
434   } else if (verdict === "likely_real") {
435     summary = `This content shows ${greenCount} positive indicator${greenCount !== 1 ? "s" : ""}...
436   } else {
437     summary = `This content has a mixed profile with ${redCount} concern${redCount !== 1 ? "s" : ""} ...
438   }
439
440   // --- REASONING GENERATION -----
441
442   const reasoningParts: string[] = [];
443
444   if (redFlags.length > 0) {
445     reasoningParts.push(`The analysis identified ${redFlags.length} concern${redFlags.length !== 1 ? "s" : ""}...
446   }
447   if (greenFlags.length > 0) {
448     reasoningParts.push(`${greenFlags.length} positive indicator${greenFlags.length !== 1 ? "s" : ""}...
449   }
450   if (reasoningParts.length === 0) {
451     reasoningParts.push("The content was too short or lacked sufficient indicators for a defini...
452   }
453   reasoningParts.push(`Overall confidence in this assessment is ${confidence}%, derived from li...
454   reasoningParts.push("This analysis is based on established media literacy frameworks used in ...
455
456   return {
457     verdict,
458     confidence,
459     summary,
460     redFlags,
461     greenFlags,
462     reasoning: reasoningParts.join(" "),
463   };
464 }
465
466 // --- MAIN ACTION -----
467
468 export const analyzeNews = action({
469   args: {
470     text: v.string(),
471     inputType: v.union(v.literal("text"), v.literal("url")),
472   },
473   handler: async (_ctx, args) => {
474     const text = args.text.trim();
475
476     if (text.length < 10) {
477       return {
478         verdict: "uncertain" as const,
479         confidence: 30,
480         summary: "The provided text is too short to perform a meaningful analysis. Please provi...
481         redFlags: ["Content is too short for reliable analysis (minimum 10 characters required)...
482         greenFlags: [],
483         reasoning: "A minimum amount of text is required to evaluate credibility indicators. Th...
484       };
485     }

```



```
486
487     return analyzeText(text);
488   },
489   });
490
```

```
1  import { authTables } from "@convex-dev/auth/server";
2  import { defineSchema, defineTable } from "convex/server";
3  import { Infer, v } from "convex/values";
4
5  // default user roles. can add / remove based on the project as needed
6  export const ROLES = {
7    ADMIN: "admin",
8    USER: "user",
9    MEMBER: "member",
10 } as const;
11
12 export const roleValidator = v.union(
13   v.literal(ROLES.ADMIN),
14   v.literal(ROLES.USER),
15   v.literal(ROLES.MEMBER),
16 );
17 export type Role = Infer<typeof roleValidator>;
18
19 const schema = defineSchema(
20   {
21     // default auth tables using convex auth.
22     ...authTables, // do not remove or modify
23
24     // the users table is the default users table that is brought in by the authTables
25     users: defineTable({
26       name: v.optional(v.string()),
27       image: v.optional(v.string()),
28       email: v.optional(v.string()),
29       emailVerificationTime: v.optional(v.number()),
30       isAnonymous: v.optional(v.boolean()),
31       role: v.optional(roleValidator),
32     }).index("email", ["email"]),
33
34     // Analysis history table
35     analyses: defineTable({
36       userId: v.string(),
37       inputText: v.string(),
38       inputType: v.union(v.literal("text"), v.literal("url")),
39       verdict: v.union(
40         v.literal("likely_real"),
41         v.literal("likely_fake"),
42         v.literal("uncertain"),
43       ),
44       confidence: v.number(),
45       summary: v.string(),
46       redFlags: v.array(v.string()),
47       greenFlags: v.array(v.string()),
48       reasoning: v.string(),
49       createdAt: v.number(),
50     }).index("by_user", ["userId", "createdAt"]),
51   },
52   {
53     schemaValidation: false,
54   },
55 );
56
57 export default schema;
58
```

```
1  import { query, mutation } from "../_generated/server";
2  import { v } from "convex/values";
3
4  export const listByUser = query({
5    args: {},
6    handler: async (ctx) => {
7      const identity = await ctx.auth.getUserIdentity();
8      if (!identity) return [];
9      const analyses = await ctx.db
10        .query("analyses")
11        .withIndex("by_user", (q) => q.eq("userId", identity.subject))
12        .order("desc")
13        .take(50);
14      return analyses;
15    },
16  });
17
18  export const get = query({
19    args: { id: v.id("analyses") },
20    handler: async (ctx, args) => {
21      const identity = await ctx.auth.getUserIdentity();
22      if (!identity) return null;
23      const analysis = await ctx.db.get(args.id);
24      if (!analysis || analysis.userId !== identity.subject) return null;
25      return analysis;
26    },
27  });
28
29  export const create = mutation({
30    args: {
31      inputText: v.string(),
32      inputType: v.union(v.literal("text"), v.literal("url")),
33      verdict: v.union(
34        v.literal("likely_real"),
35        v.literal("likely_fake"),
36        v.literal("uncertain"),
37      ),
38      confidence: v.number(),
39      summary: v.string(),
40      redFlags: v.array(v.string()),
41      greenFlags: v.array(v.string()),
42      reasoning: v.string(),
43    },
44    handler: async (ctx, args) => {
45      const identity = await ctx.auth.getUserIdentity();
46      if (!identity) throw new Error("Not authenticated");
47      return ctx.db.insert("analyses", {
48        userId: identity.subject,
49        ...args,
50        createdAt: Date.now(),
51      });
52    },
53  });
54
55  export const remove = mutation({
56    args: { id: v.id("analyses") },
57    handler: async (ctx, args) => {
58      const identity = await ctx.auth.getUserIdentity();
59      if (!identity) throw new Error("Not authenticated");
60      const analysis = await ctx.db.get(args.id);
61      if (!analysis || analysis.userId !== identity.subject) {
62        throw new Error("Not authorized");
63      }
64      await ctx.db.delete(args.id);
65    },
66  });
```

```
66     });  
67
```

```

1  import { motion } from "framer-motion";
2
3  interface CredibilityGaugeProps {
4    confidence: number;
5    verdict: "likely_real" | "likely_fake" | "uncertain";
6    size?: number;
7  }
8
9  const verdictColors: Record<string, { stroke: string; glow: string; text: string }> = {
10    likely_real: {
11      stroke: "#10b981",
12      glow: "rgba(16, 185, 129, 0.3)",
13      text: "text-emerald-600",
14    },
15    uncertain: {
16      stroke: "#f59e0b",
17      glow: "rgba(245, 158, 11, 0.3)",
18      text: "text-amber-600",
19    },
20    likely_fake: {
21      stroke: "#ef4444",
22      glow: "rgba(239, 68, 68, 0.3)",
23      text: "text-red-600",
24    },
25  };
26
27  export function CredibilityGauge({
28    confidence,
29    verdict,
30    size = 180,
31  }: CredibilityGaugeProps) {
32    const colors = verdictColors[verdict];
33    const strokeWidth = 10;
34    const radius = (size - strokeWidth) / 2;
35    const circumference = 2 * Math.PI * radius;
36    const progress = (confidence / 100) * circumference;
37    const center = size / 2;
38
39    return (
40      <div className="relative inline-flex items-center justify-center">
41        /* Glow behind */
42        <div
43          className="absolute rounded-full blur-xl"
44          style={{
45            width: size * 0.8,
46            height: size * 0.8,
47            background: colors.glow,
48          }}
49        />
50
51        <svg
52          width={size}
53          height={size}
54          viewBox={`0 0 ${size} ${size}`}
55          className="relative -rotate-90"
56        >
57          /* Background circle */
58          <circle
59            cx={center}
60            cy={center}
61            r={radius}
62            fill="none"
63            stroke="currentColor"
64            strokeWidth={strokeWidth}
65            className="text-muted/30"

```

```

66     />
67
68     { /* Animated progress arc */ }
69     <motion.circle
70       cx={center}
71       cy={center}
72       r={radius}
73       fill="none"
74       stroke={colors.stroke}
75       strokeWidth={strokeWidth}
76       strokeLinecap="round"
77       strokeDasharray={circumference}
78       initial={{ strokeDashoffset: circumference }}
79       animate={{ strokeDashoffset: circumference - progress }}
80       transition={{ duration: 1.2, delay: 0.3, ease: [0.22, 1, 0.36, 1] }}
81       style={{ filter: `drop-shadow(0 0 6px ${colors.glow})` }}
82     />
83   </svg>
84
85   { /* Center text */ }
86   <div className="absolute inset-0 flex flex-col items-center justify-center">
87     <motion.span
88       initial={{ opacity: 0, scale: 0.5 }}
89       animate={{ opacity: 1, scale: 1 }}
90       transition={{ duration: 0.5, delay: 0.6 }}
91       className="text-3xl font-extrabold tracking-tight"
92     >
93       {confidence}%
94     </motion.span>
95     <motion.span
96       initial={{ opacity: 0 }}
97       animate={{ opacity: 1 }}
98       transition={{ duration: 0.3, delay: 0.8 }}
99       className="text-[11px] text-muted-foreground font-medium"
100     >
101       confidence
102     </motion.span>
103   </div>
104 </div>
105 );
106 }
107

```

```
1  import { motion } from "framer-motion";
2  import {
3    PieChart,
4    Pie,
5    Cell,
6    BarChart,
7    Bar,
8    XAxis,
9    YAxis,
10   Tooltip,
11   ResponsiveContainer,
12 } from "recharts";
13 import {
14   TrendingUp,
15   Shield,
16   AlertTriangle,
17   CheckCircle2,
18   XCircle,
19   BarChart3,
20 } from "lucide-react";
21
22 interface Analysis {
23   verdict: "likely_real" | "likely_fake" | "uncertain";
24   confidence: number;
25   createdAt: number;
26   redFlags: string[];
27   greenFlags: string[];
28 }
29
30 interface StatsViewProps {
31   analyses: Analysis[];
32 }
33
34 const COLORS = {
35   likely_real: "#10b981",
36   uncertain: "#f59e0b",
37   likely_fake: "#ef4444",
38 };
39
40 const VERDICT_LABELS = {
41   likely_real: "Likely Real",
42   uncertain: "Uncertain",
43   likely_fake: "Likely Fake",
44 };
45
46 export function StatsView({ analyses }: StatsViewProps) {
47   if (analyses.length === 0) return null;
48
49   // Calculate stats
50   const totalAnalyses = analyses.length;
51   const verdictCounts = { likely_real: 0, uncertain: 0, likely_fake: 0 };
52   let totalConfidence = 0;
53   let totalRedFlags = 0;
54   let totalGreenFlags = 0;
55
56   for (const a of analyses) {
57     verdictCounts[a.verdict]++;
58     totalConfidence += a.confidence;
59     totalRedFlags += a.redFlags.length;
60     totalGreenFlags += a.greenFlags.length;
61   }
62
63   const avgConfidence = Math.round(totalConfidence / totalAnalyses);
64
65   const pieData = (Object.keys(verdictCounts) as Array<keyof typeof verdictCounts>).map(
```

```

66     (key) => ({
67       name: VERDICT_LABELS[key],
68       value: verdictCounts[key],
69       color: COLORS[key],
70     })),
71   ).filter((d) => d.value > 0);
72
73   // Confidence distribution
74   const confBuckets = [
75     { range: "35-50%", count: 0, fill: "#94a3b8" },
76     { range: "50-65%", count: 0, fill: "#f59e0b" },
77     { range: "65-80%", count: 0, fill: "#3b82f6" },
78     { range: "80-95%", count: 0, fill: "#10b981" },
79   ];
80   for (const a of analyses) {
81     if (a.confidence < 50) confBuckets[0].count++;
82     else if (a.confidence < 65) confBuckets[1].count++;
83     else if (a.confidence < 80) confBuckets[2].count++;
84     else confBuckets[3].count++;
85   }
86
87   const statCards = [
88     {
89       label: "Total Analyses",
90       value: totalAnalyses,
91       icon: BarChart3,
92       color: "text-primary",
93       bg: "bg-primary/10",
94     },
95     {
96       label: "Avg. Confidence",
97       value: `${avgConfidence}%`,
98       icon: TrendingUp,
99       color: "text-blue-600",
100      bg: "bg-blue-500/10",
101    },
102    {
103      label: "Red Flags Found",
104      value: totalRedFlags,
105      icon: AlertTriangle,
106      color: "text-red-600",
107      bg: "bg-red-500/10",
108    },
109    {
110      label: "Green Flags Found",
111      value: totalGreenFlags,
112      icon: CheckCircle2,
113      color: "text-emerald-600",
114      bg: "bg-emerald-500/10",
115    },
116  ];
117
118   return (
119     <div className="space-y-6">
120       {/* Stat cards */}
121       <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
122         {statCards.map((card, i) => (
123           <motion.div
124             key={card.label}
125             initial={{ opacity: 0, y: 10 }}
126             animate={{ opacity: 1, y: 0 }}
127             transition={{ duration: 0.3, delay: i * 0.08 }}
128             className="glass-card rounded-xl p-4"
129           >
130             <div className="flex items-center gap-2 mb-2">
131               <div className={`w-7 h-7 rounded-lg ${card.bg} flex items-center justify-center`}...
132                 <card.icon className={`w-3.5 h-3.5 ${card.color}`} />
133             </div>
134             <span className="text-[11px] text-muted-foreground">
135               {card.label}

```



```

136         </span>
137     </div>
138     <span className="text-xl font-bold">{card.value}</span>
139 </motion.div>
140   )}}
141 </div>
142
143 {/* Charts */}
144 <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
145   {/* Verdict distribution */}
146   {pieData.length > 0 && (
147     <motion.div
148       initial={{ opacity: 0, y: 10 }}
149       animate={{ opacity: 1, y: 0 }}
150       transition={{ duration: 0.3, delay: 0.3 }}
151       className="glass-card rounded-2xl p-6"
152     >
153       <h3 className="text-sm font-semibold text-muted-foreground uppercase tracking-wides...
154         Verdict Distribution
155       </h3>
156       <div className="flex items-center justify-center">
157         <ResponsiveContainer width={200} height={200}>
158           <PieChart>
159             <Pie
160               data={pieData}
161               cx="50%"
162               cy="50%"
163               innerRadius={55}
164               outerRadius={80}
165               dataKey="value"
166               stroke="none"
167             >
168               {pieData.map((entry) => (
169                 <Cell key={entry.name} fill={entry.color} />
170               ))}
171             </Pie>
172             <Tooltip
173               contentStyle={{
174                 background: "rgba(255,255,255,0.9)",
175                 border: "1px solid rgba(0,0,0,0.1)",
176                 borderRadius: "8px",
177                 fontSize: "12px",
178               }}
179             />
180           </PieChart>
181         </ResponsiveContainer>
182       </div>
183       <div className="flex flex-wrap justify-center gap-3 mt-2">
184         {pieData.map((d) => (
185           <div key={d.name} className="flex items-center gap-1.5">
186             <div
187               className="w-2.5 h-2.5 rounded-full"
188               style={{ background: d.color }}
189             />
190             <span className="text-[11px] text-muted-foreground">
191               {d.name} ({d.value})
192             </span>
193           </div>
194         ))}
195       </div>
196     </motion.div>
197   )}
198
199 {/* Confidence distribution */}
200 {confBuckets.some((b) => b.count > 0) && (
201   <motion.div
202     initial={{ opacity: 0, y: 10 }}
203     animate={{ opacity: 1, y: 0 }}
204     transition={{ duration: 0.3, delay: 0.4 }}
205     className="glass-card rounded-2xl p-6"

```

```
206     >
207     <h3 className="text-sm font-semibold text-muted-foreground uppercase tracking-wides...
208       Confidence Distribution
209     </h3>
210     <ResponsiveContainer width="100%" height={200}>
211       <BarChart data={confBuckets}>
212         <XAxis
213           dataKey="range"
214           tick={{ fontSize: 11, fill: "#94a3b8" }}
215           axisLine={false}
216           tickLine={false}
217         />
218         <YAxis
219           tick={{ fontSize: 11, fill: "#94a3b8" }}
220           axisLine={false}
221           tickLine={false}
222           allowDecimals={false}
223         />
224         <Tooltip
225           contentStyle={{
226             background: "rgba(255,255,255,0.9)",
227             border: "1px solid rgba(0,0,0,0.1)",
228             borderRadius: "8px",
229             fontSize: "12px",
230           }}
231         />
232         <Bar dataKey="count" radius={[6, 6, 0, 0]}>
233           {confBuckets.map((entry) => (
234             <Cell key={entry.range} fill={entry.fill} />
235           ))}
236         </Bar>
237       </BarChart>
238     </ResponsiveContainer>
239   </motion.div>
240   )}
241 </div>
242 </div>
243 );
244 }
245
```

```

1  import { motion } from "framer-motion";
2  import {
3    Brain,
4    Layers,
5    Search,
6    BarChart3,
7    Shield,
8    BookOpen,
9    Target,
10   GitBranch,
11 } from "lucide-react";
12
13 const sections = [
14   {
15     icon: Brain,
16     title: "System Architecture",
17     content:
18       "Veritas employs a multi-layered NLP analysis pipeline built on Convex (serverless backen...",
19     details: [
20       "Serverless Convex backend with real-time data synchronization",
21       "React + TypeScript frontend with Vite for fast builds",
22       "Shadcn/UI component library with custom Glassmorphism theme",
23       "Framer Motion for smooth, meaningful animations",
24     ],
25   },
26   {
27     icon: Layers,
28     title: "Three-Layer Analysis Pipeline",
29     content:
30       "The detection engine operates in three sequential layers: pattern extraction, severity s...",
31     details: [
32       "Layer 1: Regex-based linguistic pattern extraction with weighted matching",
33       "Layer 2: Category-level severity scoring (red flags vs. green flags)",
34       "Layer 3: Ratio-based verdict calculation with confidence calibration",
35       "Each layer is independently testable and auditable",
36     ],
37   },
38   {
39     icon: Search,
40     title: "Red Flag Detection (12 Categories)",
41     content:
42       "The system identifies 12 distinct categories of misinformation indicators, each with sev...",
43     details: [
44       "Sensationalist language - emotionally manipulative words and phrases",
45       "Clickbait patterns - common headline manipulation tactics",
46       "Anonymous sourcing - vague attribution without named individuals",
47       "Fear-mongering - alarmist framing designed to provoke anxiety",
48       "Conspiracy language - conspiratorial rhetoric and framing",
49       "Excessive capitalization - non-professional formatting",
50       "Emoji overuse - unprofessional content indicators",
51       "ALL CAPS emphasis - shouting via typography",
52       "Multi-exclamation marks - emotional punctuation abuse",
53       "Urgency language - pressure tactics to force sharing",
54       "Missing citations - no sources, URLs, or references",
55       "Inappropriate length - too short for genuine articles",
56     ],
57   },
58   {
59     icon: Target,
60     title: "Green Flag Detection (9 Categories)",
61     content:
62       "Simultaneously, the system evaluates credibility indicators consistent with established ...",
63     details: [
64       "Named sources with credentials and specific attributions",
65       "Quantitative data - statistics, percentages, dollar amounts",

```

```

66     "Credible institutions - Reuters, BBC, universities, journals",
67     "Temporal specificity - exact dates, timelines, event markers",
68     "Balanced reporting - multiple perspectives, counterarguments",
69     "Journalistic structure - who, what, where, when, why framework",
70     "Neutral tone - absence of emotional or manipulative language",
71     "Appropriate length - 80-800 words, consistent with news articles",
72     "Named quotes - direct attributions to specific individuals",
73   ],
74 },
75 {
76   icon: BarChart3,
77   title: "Scoring Methodology",
78   content:
79     "Verdicts are determined by calculating the ratio of red flag severity scores to green fl...
80   details: [
81     "Red flag ratio ? 0.65 <> Likely Fake (confidence 55-95%)",
82     "Green flag ratio ? 0.65 <> Likely Real (confidence 55-95%)",
83     "Ratio difference > 0.1 <> Leaning verdict (confidence 42-78%)",
84     "Balanced scores <> Uncertain (confidence 35-50%)",
85     "Confidence is calibrated to prevent overconfident claims",
86   ],
87 },
88 {
89   icon: GitBranch,
90   title: "Why Rule-Based Over ML?",
91   content:
92     "The system deliberately uses rule-based NLP rather than trained ML models. This design d...
93   details: [
94     "Explainability - every decision can be traced to specific patterns",
95     "Transparency - users see exactly why content was flagged",
96     "No training data required - works immediately without labeled datasets",
97     "Interpretability - critical for educational and academic applications",
98     "Consistency - deterministic output for the same input",
99     "Trade-off acknowledged: ML models (BERT, RoBERTa) could achieve higher accuracy but sacr...
100   ],
101 },
102 {
103   icon: BookOpen,
104   title: "Academic References",
105   content:
106     "The detection heuristics are informed by established misinformation research:",
107   details: [
108     "MIT Media Lab - Fake News detection using linguistic feature extraction",
109     "First Draft News - Misinformation taxonomy and indicator framework",
110     "Stanford Internet Observatory - Virality Project pattern analysis",
111     "Reuters Institute - Digital News Report credibility framework",
112     "LIAR Dataset (Wang, 2017) - 12.8K labeled statements for NLP research",
113     "MediaWise - Teen media literacy indicators research",
114   ],
115 },
116 {
117   icon: Shield,
118   title: "Limitations & Future Work",
119   content:
120     "The current approach has acknowledged limitations that could be addressed in future vers...
121   details: [
122     "Does not cross-reference claims against external fact-check databases",
123     "No image/video analysis (only text-based content)",
124     "Rule-based system requires manual pattern updates",
125     "Future: Fine-tuned BERT classifier on LIAR dataset (12.8K statements)",
126     "Future: TF-IDF vectorization for topic modeling",
127     "Future: Google Fact Check Tools API integration",
128     "Future: Image forensics for manipulated media detection",
129   ],
130 },
131 ];
132
133 export function MethodologyView() {
134   return (
135     <div className="space-y-6">

```

```

136      {/* Header */}
137      <motion.div
138        initial={{ opacity: 0, y: 10 }}
139        animate={{ opacity: 1, y: 0 }}
140        className="glass-card rounded-2xl p-6"
141      >
142        <div className="flex items-center gap-3 mb-3">
143          <div className="w-10 h-10 rounded-xl bg-primary/10 flex items-center justify-center">
144            <BookOpen className="w-5 h-5 text-primary" />
145          </div>
146          <div>
147            <h2 className="text-lg font-bold">Technical Methodology</h2>
148            <p className="text-sm text-muted-foreground">
149              How Veritas detects misinformation - for academic reference
150            </p>
151          </div>
152        </div>
153      </motion.div>
154
155      {/* Sections */}
156      {sections.map((section, i) => (
157        <motion.div
158          key={section.title}
159          initial={{ opacity: 0, y: 10 }}
160          animate={{ opacity: 1, y: 0 }}
161          transition={{ duration: 0.3, delay: i * 0.06 }}
162          className="glass-card rounded-2xl p-6"
163        >
164          <div className="flex items-center gap-3 mb-3">
165            <div className="w-9 h-9 rounded-xl bg-primary/10 flex items-center justify-center s...
166              <section.icon className="w-4 h-4 text-primary" />
167            </div>
168            <h3 className="text-base font-bold">{section.title}</h3>
169          </div>
170          <p className="text-sm text-muted-foreground leading-relaxed mb-3">
171            {section.content}
172          </p>
173          <ul className="space-y-1.5">
174            {section.details.map((detail) => (
175              <li
176                key={detail}
177                className="flex items-start gap-2 text-xs text-muted-foreground leading-relaxed...
178              >
179                <div className="w-1 h-1 rounded-full bg-primary mt-1.5 shrink-0" />
180                {detail}
181              </li>
182            ))}
183          </ul>
184        </motion.div>
185      ))}
186    </div>
187  );
188 }
189

```

Veritas

AI-Powered Misinformation Detection System

BSc Data Science Third Year Project - August 2026